Development of AI Response Validation System with Hallucination Detection Assistance

EvalAI - AI Response Quality Evaluator

Project Report

Prepared for: Infosys Springboard Virtual Internship 7.0

Author: Ghayathri Devi G M

1. Introduction

As AI-generated responses become a routine part of how information is delivered, the trustworthiness of those responses cannot be assumed. A response can be fluent and confident while still being irrelevant, factually wrong, incomplete, or partly fabricated. This project, the AI Response Quality Evaluator Engine, was built to address this gap by evaluating AI responses automatically, across multiple distinct quality dimensions, and grounding those evaluations in retrieved reference material rather than a single model's unchecked opinion.

The system was developed iteratively across four milestones: an initial RAG-grounded prototype, the addition of three independent LLM-as-judge agents, the addition of a fourth agent and an aggregating Verdict Agent alongside batch processing, and finally an analytics dashboard, PDF reporting, and end-to-end validation. This report documents that full development effort.

1.1 Scope and Intended Use

The platform is designed for developers, QA reviewers, or content teams who need to check AI-generated answers against a known question set — either one response at a time during development, or in bulk against a benchmark CSV before a release. It is not designed to generate answers itself; it strictly evaluates answers that already exist, whether produced by the same system being tested, a third-party model, or a human reviewer manually checking a candidate response.

The current scope assumes a single trusted user per deployment (no authentication layer), a knowledge base scoped to SQuAD and a slice of Wikipedia, and a free-tier LLM provider (Groq) as the reasoning engine behind each judge agent.

2. Objectives

- Ground evaluation in real reference material using a Retrieval-Augmented Generation (RAG) pipeline, rather than judging responses in isolation.
- Build independent judge agents for Relevance, Accuracy, Completeness, and Hallucination Detection, each producing a score and explicit reasoning.
- Aggregate the four agents into a single weighted verdict that reflects the relative importance of factual correctness and grounding over surface relevance.
- Support both single-response and bulk CSV batch evaluation.
- Persist evaluation history and provide a scoring dashboard for tracking quality trends.
- Provide structured, exportable PDF reports summarizing batch results.
- Validate the system end-to-end and document its behaviour, limitations, and defects honestly.

3. Methodology

Development followed an Agile process across four sprints, each aligned to one milestone. Work was tracked using a Product Backlog, Sprint Backlog, daily stand-up logs, sprint retrospectives, and a defect tracker, maintained as Excel artifacts alongside the codebase.

3.1 Sprint 1 — Foundation

Research into RAG architecture and existing evaluation frameworks (RAGAS, TruLens), followed by system architecture design, the Evaluation Input Module, and a Reference Knowledge Base built from SQuAD and a slice of Wikipedia, indexed in ChromaDB.

3.2 Sprint 2 — Judge agents

Built the Relevance, Accuracy, and Hallucination Detection agents as independent LLM-as-judge modules using Groq, migrated evaluation history from SQLite to Supabase for persistence beyond local development, and validated all three agents against SQuAD benchmark pairs.

3.3 Sprint 3 — Completeness, Verdict, and batch processing

Added the Completeness Judge Agent, the Verdict Agent (weighted aggregation), the ability to submit reference answers and PDFs optionally, and the Batch Evaluation Module for CSV-based bulk processing with concurrent per-row agent execution.

3.4 Sprint 4 — Analytics, reporting, and validation

Built the Evaluation Scoring Dashboard with filterable analytics and charts, the PDF report export feature, and a formal end-to-end test suite covering all system workflows, agent scoring, and error handling.

Throughout all four sprints, defects were logged as they were discovered during real testing — rather than only in a final QA pass — and fixes were verified before the next sprint began. This is reflected in the Testing section below.

4. System Design

The system follows a layered architecture: an input layer (single-submission form or CSV upload), a retrieval layer (ChromaDB vector search), four parallel evaluation agents, an aggregation layer (the Verdict Agent), a persistence layer (Supabase), and two presentation surfaces (the analytics dashboard and PDF export).

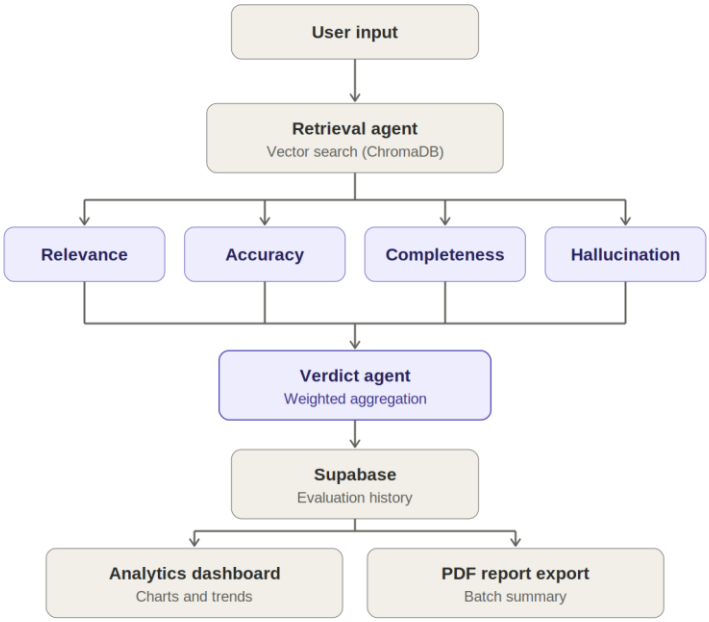

Figure 1. System architecture — input, retrieval, four parallel judge agents, verdict aggregation, persistence, and outputs.

Two entry points feed the identical pipeline: the single-evaluation endpoint and the batch CSV endpoint, which runs each row through the same four agents (executed concurrently per row for performance) before persisting results.

4.1 Database schema

A single Supabase (PostgreSQL) table, evaluations, stores every evaluation from both single and batch modes, including each agent's score, reasoning, and any flagged claims or missing aspects, plus the aggregated verdict and an evaluation_mode tag distinguishing single from batch submissions.

4.2 Orchestration flow

The pipeline is not fully sequential. Preprocessing and retrieval must complete before evaluation can begin, since every agent depends on the retrieved context being ready. Once retrieval finishes, however, the four judge agents

have no dependency on one another's output — Relevance only needs the question and answer, Accuracy and Hallucination Detection need the answer plus retrieved context, and Completeness only needs the question and answer. Because of this independence, all four agents are dispatched concurrently via a thread pool rather than one after another, and the pipeline waits (a synchronization join) until all four have returned before the Verdict Agent runs. This distinction — sequential where a genuine dependency exists, parallel where none does — was applied consistently across both the single-evaluation and batch-evaluation code paths, and was the direct fix for a performance defect discovered during batch testing (Section 9.3).

5. Implementation

5.1 Retrieval-Augmented Generation

A one-time indexing script (knowledge_builder.py) chunks SQuAD and Wikipedia text into 100-word segments, embeds them with Sentence-Transformers (all-MiniLM-L6-v2), and stores them in ChromaDB. At request time, the incoming question is embedded and the top-k nearest chunks are retrieved, optionally supplemented by text extracted from an uploaded PDF.

5.2 Judge agents

Each agent (relevance_agent.py, accuracy_agent.py, completeness_agent.py, hallucination_agent.py) makes a single call to Groq's llama-3.1-8b-instant model with a structured system prompt requiring a JSON response containing a score and reasoning. Every agent includes a local embedding-similarity fallback if the LLM call fails twice, and a retry-with-repair step (json_repair) for malformed JSON output — a defect found during real testing and fixed at the parsing layer rather than relying on prompt instructions alone.

The Hallucination Detection Agent includes an additional grounding safety net: every claim it flags is re-verified against the raw source text using exact and fuzzy string matching, automatically correcting cases where the judge model contradicts itself.

5.3 Verdict Agent

Implemented as pure aggregation logic with no LLM call. It combines the four agents' scores using fixed weights (Accuracy 35%, Hallucination 30%, Relevance 20%, Completeness 15%) and maps the result to a verdict label (Excellent, Good, Needs Improvement, or Poor).

5.4 Batch processing

batch_evaluator.py parses an uploaded CSV, validates required columns, caps batch size at 20 rows to bound API usage, and evaluates each row through the same four-agent pipeline. The four per-row agent calls run concurrently via a thread pool, since they are independent of each other's output — a performance fix applied after early batch runs were found to be running all calls sequentially.

5.5 Analytics and reporting

The analytics endpoint aggregates stored evaluations into pass/needs-improvement/fail counts, per-dimension averages, hallucination frequency, and a day-by-day trend, filterable by date range and evaluation mode. The PDF export feature (report_generator.py) formats already-computed batch results into a structured report using reportlab and matplotlib, making no additional LLM calls.

6. Experimental Results

The system was exercised with a range of test cases designed to probe each agent's behaviour, including deliberately correct and deliberately incorrect answers on topics confirmed to be present in the indexed knowledge base.

6.1 Representative case: multi-claim mixed answer

A question about Notre Dame's history and architecture was answered with a response containing one plausible claim and two fabricated claims (an invented founding date and a false enrollment figure). The system correctly separated the claims:

Figure 2. Per-dimension scores for a mixed-claim answer — accuracy 0%, relevance 20%, hallucination risk High, with each fabricated claim flagged individually.

All three unsupported claims were flagged individually and by name, rather than the system returning only an aggregate "this response is bad" judgment — demonstrating claim-level detection rather than holistic guessing.

6.2 Batch evaluation results

A five-row batch spanning correct, incorrect, and partially-correct answers produced a spread of verdicts consistent with each row's actual quality, confirming that scoring differentiates real quality differences rather than clustering around a default value:

Figure 3. Batch evaluation per-row results, showing a spread of verdicts (Excellent, Good, Needs Improvement) matching the intended quality of each test row.

6.3 Aggregate scores across the batch

The table below summarizes the same batch run's aggregate statistics, computed by analytics.py from the five individual results:

Metric	Value	Interpretation
Rows evaluated	5	Full batch processed without error
Average accuracy	72%	Mixed batch, weighed down by one deliberately wrong answer
Average relevance	76%	Most answers stayed on-topic
Average completeness	78.6%	Most answers covered the required aspects of their question
Verdict distribution	2 Good, 1 Excellent, 2 Needs Improvement	Distinct verdicts per row, not a single repeated label

Hallucinated claims flagged	2	Both from the row containing a deliberately fabricated answer

The spread across verdict labels (rather than every row landing on the same score) is itself evidence that the four agents are producing genuinely differentiated judgments based on each answer's actual content, not a fixed or default output.

7. Dashboard Screenshots

The following screenshots were captured during real testing sessions and reflect the actual running system, not mockups.

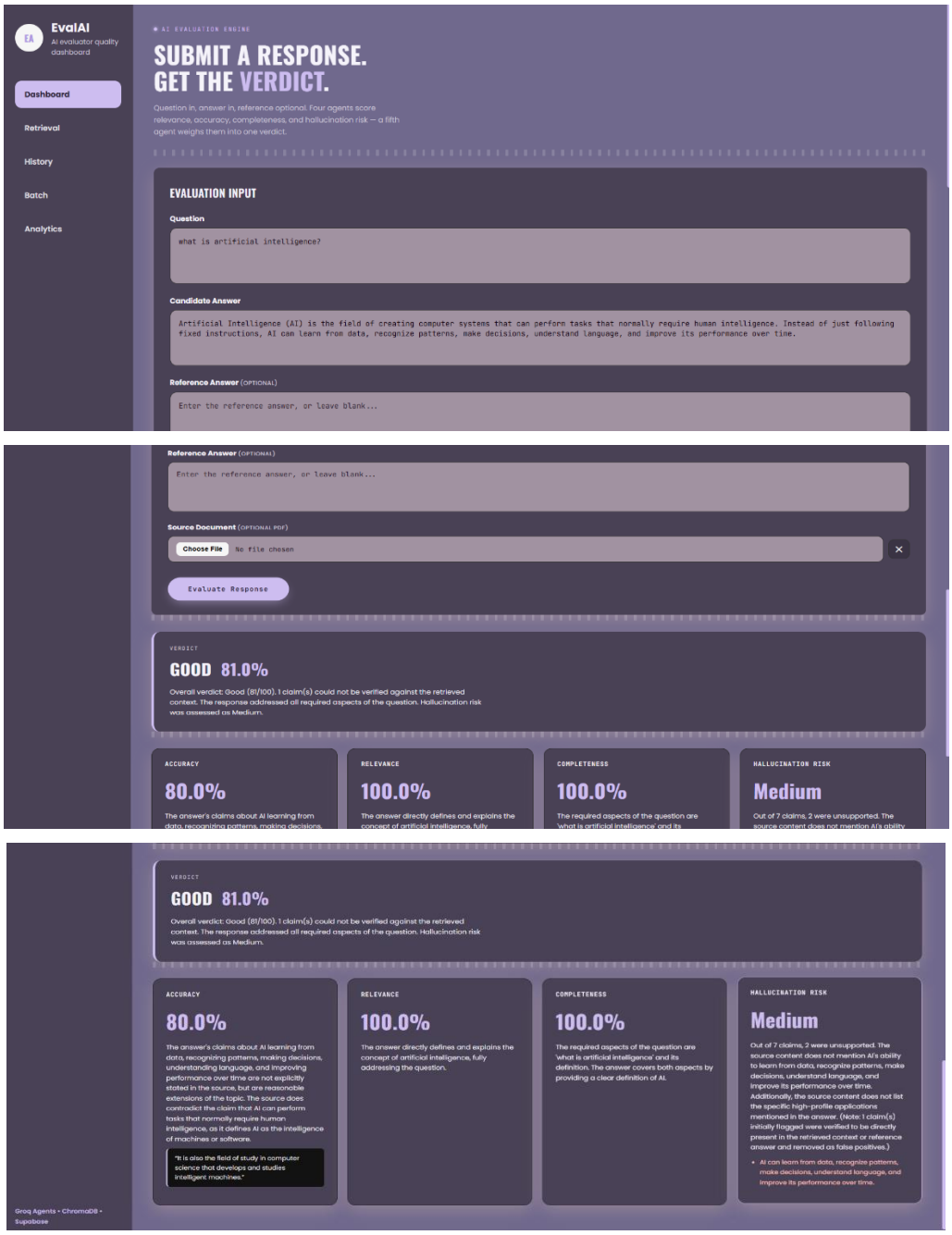

Figure 4. Evaluation Input — question, candidate answer, and optional reference answer / source PDF.

Figure 5. Aggregated Analytics

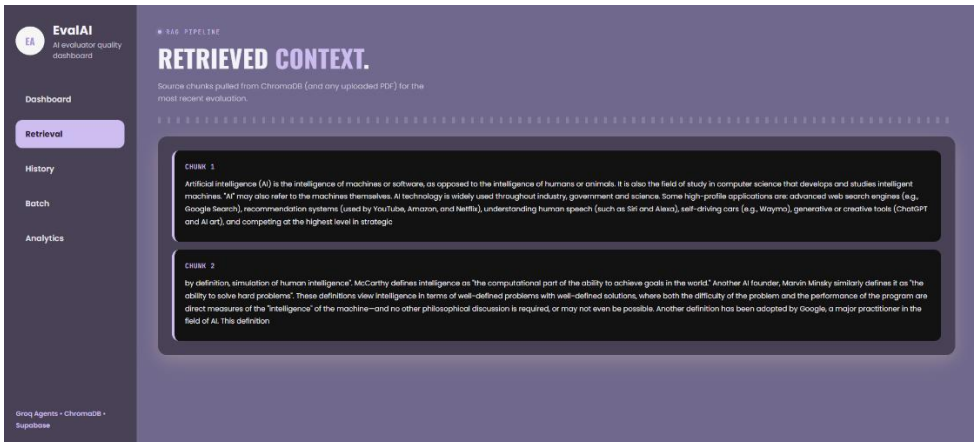

Figure 6. Retrieved contexts from knowledge base

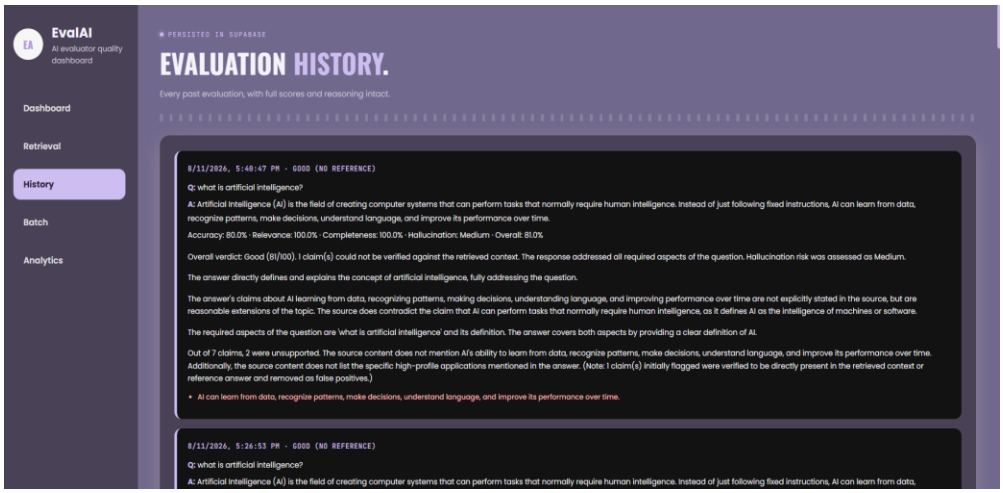

Figure 7. Evaluation history

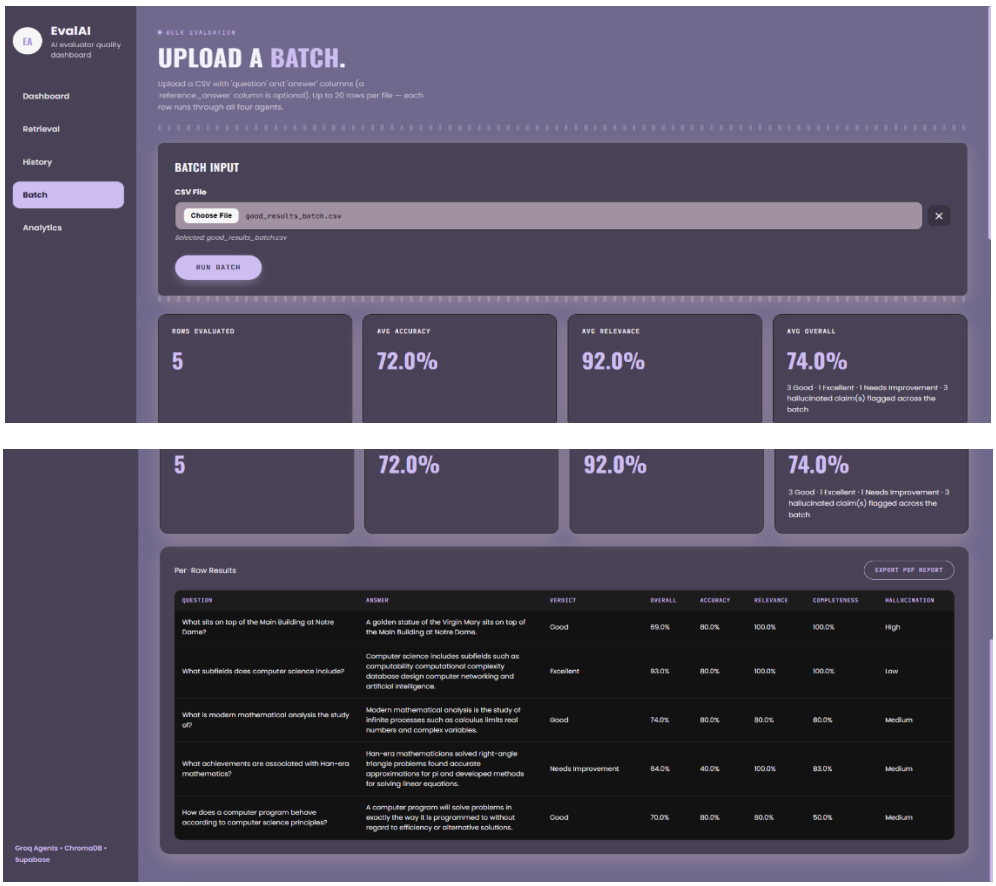

Figure 8. Batch evaluation

8. Evaluation Analysis

The Verdict Agent's weighting (Accuracy 35%, Hallucination 30%, Relevance 20%, Completeness 15%) was a deliberate design choice rather than an equal average: a response that is relevant and thorough but factually wrong or

fabricated is judged more harshly than one that is accurate and grounded but only partially complete. This reflects the intended use case — catching untrustworthy content — over rewarding superficial thoroughness.

8.1 Worked example

For the mixed-claim response shown in Figure 2 (accuracy 0%, relevance 20%, hallucination risk High, completeness assumed unscored in that isolated test), the weighted formula computes as:

overall_score = (relevance × 0.20) + (accuracy × 0.35) + (completeness × 0.15) + (hallucination_proxy × 0.30)

With a High hallucination risk mapped to a proxy value of 0.2, and completeness held at a neutral value for illustration, the formula yields a low overall score and a "Poor" verdict label — correctly reflecting that the response's severe accuracy and grounding failures dominate the result even though relevance was not zero. An equal four-way average of the same inputs would have produced a noticeably higher, more forgiving score, which is precisely the outcome the weighting was designed to avoid.

8.2 Accuracy scoring recalibration

The Accuracy Agent's scoring was recalibrated during development after an early version treated any unstated claim as equivalent to a false one, producing 0% scores for reasonable, uncontradicted statements. The revised scoring distinguishes CONTRADICTED claims (scored near zero) from UNSTATED-but-reasonable claims (given partial credit), which is a more defensible standard of "accuracy" than strict verbatim matching against a limited reference source.

8.3 Retrieval-dependent false positives

A known limitation surfaced during batch testing: general-knowledge questions outside the indexed SQuAD/Wikipedia dataset can cause retrieval to return irrelevant context, which previously caused the Hallucination Agent to falsely flag correct answers. This was corrected by having the agent check claims against the reference answer in addition to retrieved context, rather than relying on retrieval alone.

9. Testing

9.1 Agent validation

validate_agents.py benchmarks all agents against SQuAD question-answer pairs, each tested with both a correct answer and a deliberately wrong answer (a false claim injected into the correct one). The suite checks two properties: scoring consistency (correct answers should score higher than wrong answers, on average) and reasoning quality (every agent response must include substantive, non-empty reasoning).

9.2 End-to-end testing

e2e_test.py exercises the full running system across nine required coverage areas, combining live HTTP requests against the running server (for workflow-level tests) with direct function calls (for agent-level and verdict-math tests, which do not require the server to be running):

#	Coverage area	Method
1	Single evaluation workflow	Live POST /evaluate call
2	Batch evaluation workflow	Live POST /batch-evaluate with a test CSV
3	Dashboard updates	Compares /analytics totals before and after a new evaluation
4	Report generation	Live POST /export-report, verifies a valid PDF is returned

5	RAG retrieval	Direct call to retrieve_context()
6	Agent scoring	Direct calls to all four judge agents
7	Verdict generation	Verifies the weighted formula and label banding are exact
8	Error handling	Malformed CSV and oversized batch — expects clean 400s
9	Invalid input handling	Missing fields, empty strings, wrong file type

9.3 Defects found and resolved

- Accuracy agent's LLM output occasionally broke JSON parsing on longer answers — resolved with a tolerant json_repair fallback and a shortened output-length requirement.
- Hallucination agent produced false positives when retrieval missed relevant chunks — resolved by also checking claims against the reference answer.
- Batch evaluation results were not being persisted to Supabase at all in an early version — resolved by wiring save_evaluation() into the batch pipeline.
- Batch row processing was slow due to sequential agent calls — resolved by parallelizing the four independent per-row calls with a thread pool.

9.4 Pass criteria

A test is considered passing only when its specific, pre-defined condition is met exactly — for example, the verdict generation test does not simply check that a score was returned, but recomputes the expected weighted value independently and requires an exact match. Error-handling tests require a specific 400-class response, not merely the absence of a crash. This stricter bar was chosen deliberately over looser “did it run without an exception” checks, since the latter would not have caught the batch-persistence defect described above, which ran without error but silently failed to save data.

10. Conclusion

The EvalAI - AI Response Quality Evaluator successfully demonstrates a working, RAG-grounded, multi-agent evaluation pipeline capable of judging AI responses across four distinct quality dimensions, aggregating them into a weighted, human-readable verdict, and scaling from single submissions to bulk CSV batches with persistent history, analytics, and exportable reporting.

Development surfaced genuine engineering challenges — inconsistent LLM JSON output, retrieval gaps causing false hallucination flags, and a data-persistence gap in the batch pipeline — each of which was identified through real testing and resolved with a defensible fix rather than a workaround. The system's known limitations (a lightweight judge model, a bounded knowledge base, fixed verdict weights, and no multi-user support) are documented honestly rather than concealed, and are the basis for the future work items already scoped for continued development.

Overall, the project met its stated objectives: it grounds evaluation in real reference material, produces reasoned rather than opaque judgments, and provides the persistence, analytics, and reporting needed to use the tool for ongoing quality tracking rather than one-off checks.

The project's four-milestone structure meant the system was never built as a single monolithic effort but grew deliberately in scope — from a grounded prototype, to independent judge agents, to weighted aggregation and batch processing, to analytics and formal validation — with each stage's defects surfaced and fixed before the next began. That incremental discipline, more than any single feature, is what made the final system's behaviour explainable and its limitations known rather than hidden.